A smaller guarded presentation of iPRA

Combining deterministic introduction with source-selected elimination

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The two reductions called *guarded core* and *source dispatch* in ConciseIPRA can be used together. The mathematical point is a dependency theorem: literal-zero-source transport is already admissible in a purely equational fragment with contextual equation induction. Its proof does not need logical case introduction. This breaks the apparent circularity between the two reductions.

The resulting presentation is bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Both numerical domains are whole, equality is actual equality, and both numerical composites have the identity graph. The public formulas have the existing guarded grammar; proof trees may contain the explicitly specified auxiliary formulas. Three complete compact checker profiles attain 257 significant tokens, 279 AST nodes, and 522 normalized bytes, respectively, improving the pinned coordinate minima 259, 283, and 526. These are separate programs. A standalone version that also validates the closed public-goal grammar is 1,202 normalized bytes, versus 1,208 for the corresponding guarded-core validator and kernel. The gain is strict but small. No absolute minimality or new proof-assistant verification is claimed.

1 Result, baseline, and comparison contract

The comparison uses the repository snapshot

`1517d9d79ce33f631e0f82960c849427cbba23bf`

under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The pinned README, the relevant source files, and the measured baseline checkers are supplied in the archive. The snapshot already includes the Lean theorem `zprf_iff_nprf`, the direct-closure rule, and the `guarded_core` and `source_dispatch` research packages [1, 3, 2, 5]. Those individual constructions are inherited.

The contribution here is their compatible composition, supported by the equational dependency theorem below. A small implementation change also replaces a parity test on array length by the exact equality-root test. The mathematical argument and that implementation change are distinguished throughout.

Complete module	Tokens	AST nodes	Bytes	Gzip-9 bytes
Pinned source-dispatch, token profile	259	289	572	339
Proposed token profile	**257**	285	568	338
Pinned source-dispatch, AST profile	263	283	607	348
Proposed AST profile	261	**279**	603	345
Pinned source-dispatch, byte profile	266	305	526	330
Proposed byte profile	264	301	**522**	328

Each new profile improves all three primary coordinates of its corresponding source-dispatch profile. The minima across the three new modules are $(257, 279, 522)$; no single module in this package attains that triple. The old minima are attained results, not proved lower bounds. Gzip is reported as information, not an optimization objective.

The compact profiles use the guarded-core convention: the public theorem being checked is a canonical guarded formula, while certificates may use a larger auxiliary language. They do not reject every noncanonical end formula. For applications that require that rejection, the fully counted standalone strict checker is also supplied:

Goal validator and kernel	Tokens	AST nodes	Bytes	Gzip-9 bytes
Pinned guarded-core strict checker	491	568	1,208	574
Proposed strict checker	490	561	1,202	571

The 522-byte figure does not include this public-grammar validator.

2 The complete proposed presentation

2.1 Terms, public formulas, and auxiliary formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in h; it is not bound in n or a. The body h may contain fixed numerical parameters. Terms do not contain a numerical variable ranging over program descriptions, and no uniform evaluator is postulated. Write $1 = S0$, $2 = S1$, and

$$C(v, a, b) := I(v, a; u.b),$$

where the update variable is unused. All binders use capture-free substitution and may be renamed.

Definition 2.1 (Public language)**.** A canonical public formula is a literal equation $s = t$, or a formula $D_{x,y}B$ satisfying one of the following two clauses:

1. B is a compound canonical formula, so its root is D;
2. B is exactly of the guarded atomic shape

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{G}$$

The bodies h, k may depend on x, y, u and passive parameters. In particular, the guard is a syntactic condition, not a test for the theoremhood of a base case. Both sides in (G) compute to zero at $y = 0$. The designated contradiction is the public equation $0 = 1$.

The finite auxiliary formula language used *inside proofs* is larger:

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A. \tag{A}$$

It permits unguarded atomic matrices. A retained context is a finite reusable list of auxiliary formulas. A public sequent has a canonical context and canonical conclusion, but its proof may use (A) internally. This convention is part of the construction, not an omitted restriction. There is no claim below about forcing every intermediate formula to satisfy the public guard.

2.2 All primitive axioms and rules

In every context the two nonlogical axiom schemas are

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

There are exactly the following four inference patterns besides those two computation schemas.

Hypothesis.

$$\frac{}{\Gamma \vdash A} \qquad (A \in \Gamma). \tag{Hyp}$$

Elimination selected by the source. For any fixed auxiliary formula template $F(v)$, put

$$M(s, t; F) := \begin{cases} Q_v F(v), & s \equiv 0, \\ s = t, & s \not\equiv 0. \end{cases}$$

The rule is

$$\frac{\Gamma \vdash M(s, t; F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{SE}$$

Here $s \not\equiv 0$ means that the source is not literally the one-node term 0. A variable or an iterator that evaluates to zero is still a permitted equality source. The $s \equiv 0$ branch is ordinary anchored Q elimination; the other branch is restricted equality transport.

Paired elimination.

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenparameter b is fresh for Γ, A, and the passive parameters of B.

One deterministic paired introduction. Define $\eta_B(z) = z$ when B is a literal equation and $\eta_B(z) = 0$ otherwise. Then

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}. \tag{DI}$$

The eigenparameter z is fresh for Γ, t, and the passive parameters of B. The local assumption is discharged. There is no certificate choice between cases and progress: the root of the fixed matrix determines it. Atomic matrices use progress; compound matrices use their zero instance.

Call this auxiliary proof calculus $\mathcal{U}^-$, and call its consequence relation on public canonical sequents the proposed presentation. There are no additional primitive axioms or rules for reflexivity, symmetry, successor injectivity, successor separation, numerical coverage, induction, propositional connectives, or ordinary quantifiers. Weakening, assumption cut, and fresh renaming are admissible by induction on derivations. Arbitrary numerical substitution into a restricted proof is not assumed in the bootstrap argument.

3 Soundness and the purpose of the guard

Interpret the terms in iPRA by their primitive recursive functions, one fixed finite iterator description at a time. Equality is unchanged. Give auxiliary formulas the following compositional reading, denoted by a superscript $*$:

$$\begin{aligned}(s=t)^* &= (s=t),\\ (Q_y B)^* &= B^*(0) \to \forall y\, B^*(y),\\ (D_{x,y}B)^* &= \begin{cases}\exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ a literal equation},\\ \exists x\, (B^*(x,0) \to \forall y\, B^*(x,y)), & B \text{ compound}.\end{cases}\end{aligned}$$

This is a specified syntax-sensitive translation. It is not an assertion that a single ordinary connective has interchangeable meanings when its matrix is replaced by a logically equivalent formula.

Lemma 3.1 (Auxiliary soundness)**.** *If $\Gamma \vdash_{\mathcal{U}^-} A$, iPRA proves the corresponding consequence from Γ^* to A^*.*

Proof. Computation and hypotheses are immediate. Numerical substitution preserves the root constructor of a formula, so the reading commutes with numerical substitution; the equality branch of SE is sound. Its zero branch is anchored Q elimination.

At an atomic DI, the premise proves each progressive implication, and universal introduction followed by existential introduction proves the displayed atomic reading. At compound DI, numerical coverage supplies the missing zero-or-successor split, yielding the anchored reading.

At an atomic DE, a progressive equation predicate gives $E(0) \to \forall y\, E(y)$ by contextual equation induction in iPRA. This is exactly the exported Q resource. Compound DE is ordinary existential elimination. All eigenvariable conditions are the standard ones. Induction on the finite derivation completes the proof. □

The public ordinary reading, denoted by A°, instead reads every public D uniformly as

$$(D_{x,y}B)^\circ = \exists x\, (B^\circ(x,0) \to \forall y\, B^\circ(x,y)). \tag{O}$$

Lemma 3.2 (Agreement on the public grammar)**.** *For every canonical formula A, iPRA proves $A^* \leftrightarrow A^\circ$.*

Proof. At a guarded atomic matrix $E(x,y)$, computation proves $E(x,0)$. Consequently, with x fixed,

$$\forall y\, (E(x,y) \to E(x,Sy)) \quad \longleftrightarrow \quad E(x,0) \to \forall y\, E(x,y).$$

The forward implication is equation induction; the reverse uses the theorem $E(x,0)$ to obtain all instances. Existential quantification preserves the equivalence. At a compound matrix the two readings have the same form, so structural induction applies. □

The guard is essential to this argument. Without a true zero instance, a progressive predicate and an anchored predicate need not agree. For example, the equation asserting that the parity iterator is 1 is false at zero, true at one, and false at two. Its anchored closure is true, while its progressive closure is false. We make no completeness or composite claim for all unguarded auxiliary formulas.

4 The equational dependency theorem

This section isolates a smaller set of hypotheses for the repository's literal-zero-source argument [2]. The singleton test and the zero-filter construction are inherited from that argument. The dependency statement, and its use in the combined presentation, are the point here.

Definition 4.1 (Equational fragment)**.** Let $\mathcal{E}^-$ have only equation formulas, hypotheses, the two computation schemas, equation-template transport

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad (s \not\equiv 0,\ F \text{ a literal equation template}), \tag{Tr$^-$}$$

and contextual equation induction

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(r)}. \tag{EI}$$

The eigenparameter z is absent from $\mathrm{FV}(\Gamma)$, the target r, and all passive parameters of E; these remain fixed during induction.

Theorem 4.2 (Equational source elimination)**.** *Unrestricted equation-template transport is admissible in $\mathcal{E}^-$. Thus the restriction $s \not\equiv 0$ changes neither its closed equational theorems nor its contextual equational consequences. No logical binder or logical case introduction is needed.*

4.1 Equation cases and safe equality

Equation cases already follow from EI: given $E(0)$ and $E(Sz)$ in a fixed context, weaken the successor proof by the unused assumption $E(z)$ and apply EI at the desired target. This observation removes the logical case-introduction dependency from the original proof.

Reflexivity of an arbitrary r follows by taking $b = I(0, r; u.0)$ and using $b = r$ as both transport premises with motive $v = r$. The source b is allowed. From $p = q$ with $p \not\equiv 0$, permitted transport gives both $q = p$ and $H(p) = H(q)$ for every numerical template H. All computation equations have an iterator on the left and can therefore be reversed as equations. Their reversed equations can be used as transport majors only when the new left side is syntactically nonzero.

Lemma 4.3 (Joining through a represented zero)**.** *If $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$, then Γ proves $p = q$ using computation and restricted transport alone.*

Proof. Put $z_0 = I(0, 0; u.0)$ and $b_p = I(0, p; u.0)$. Transport $b_p = b_p$ along $p = 0$ in the left-hand occurrence of its initializer, obtaining $z_0 = b_p$. Reverse at the permitted source z_0, then use $b_p = p$ to get $p = z_0$. Similarly derive $q = z_0$. Reverse the latter at source q and transport through z_0 to obtain $p = q$. Every source used is one of p, q, z_0, b_p, b_q, hence is permitted. This avoids transitivity through literal zero before that operation has been established. □

4.2 Two unconditional tests

Define the following finite scalar terms:

$$\begin{aligned} Z(v) &= C(v, 1, 0), & K(v) &= C(v, 0, 1), & H(v) &= C(v, 1, 2), \\ N(n) &= I(n, 0; u.Z(u)), & R(n) &= I(n, 0; u.H(u)), & L(n) &= N(R(n)). \end{aligned}$$

These are explicit abbreviations over $0, S, I$, not new function symbols.

Lemma 4.4 (Test and filter identities)**.** *The equational fragment proves, at every term target v,*

$$L(1) = 1, \qquad L(SSv) = 0, \qquad L(Sv) = Z(v), \tag{T}$$

and, at every term target t,

$$t = C(Z(t), t, 0). \tag{F}$$

Proof. Equation cases prove $H(v) = SK(v)$: the zero endpoints compute to 1 and the successor endpoints compute to 2. These joins use the nonzero numerals $1, 2$. Computation gives $N(1) = 1$, $N(2) = 0$, and $R(1) = 1$. Also

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Replacing the iterator source $R(Sv)$ in $R(SSv) = H(R(Sv))$ now gives $R(SSv) = 2$. Hence $L(1) = 1$ and $L(SSv) = 0$ by replacement at iterator sources inside N.

For $L(Sv) = Z(v)$, use equation cases. At zero both endpoints equal 1. At a successor both equal zero, and both left-hand expressions are iterator terms, so the preceding joining lemma applies. The case rule itself supplies every arbitrary target; no substitution into a proof with a variable transport source is required.

For the filter, apply equation cases to $v = C(Z(v), v, 0)$. At zero the right side computes to $C(1, 0, 0) = 0$; at Sv it computes to $C(0, Sv, 0) = Sv$. Reverse these equations at their iterator sources to obtain the required branch orientations. Both branch contexts are empty. □

4.3 Reversing an equality whose source is zero

Lemma 4.5 (Zero-source symmetry)**.** *For every term t, $0 = t \vdash_{\mathcal{E}^-} t = 0$.*

Proof. If $t \equiv 0$ syntactically, use reflexivity. Otherwise fix t and retain the assumption $0 = t$. Put $A(n) = I(n, t; u.Su)$ and induct on

$$E(n) \equiv n = A(n).$$

At the base, computation gives $A(0) = t$, and safe symmetry gives $t = A(0)$. Transport in the retained $0 = t$ at the permitted source t gives $0 = A(0)$.

At the step, $z = A(z)$ has a variable source, which is permitted. Successor congruence gives $Sz = SA(z)$. Reverse the computation $A(Sz) = SA(z)$ and transport at source $SA(z)$ to obtain $Sz = A(Sz)$. The parameter t and the context remain fixed.

Apply EI directly at 1. It gives $1 = A(1)$; forward computation of $A(1)$ gives $1 = St$. Congruence for L, followed by (T), gives $1 = Z(t)$; safe symmetry at source 1 gives $Z(t) = 1$. Weaken the unconditional filter (F) into the retained context. Replace $Z(t)$ by 1 there, then compute $C(1, t, 0) = 0$. These last sources are iterators, and the result is $t = 0$. □

Completion of Theorem 4.2. Full symmetry now follows by the safe-source case and the preceding lemma. For congruence from $0 = t$ with $t \not\equiv 0$, first reverse it, use permitted congruence at t, and reverse the resulting equality. The literal-zero/literal-zero case uses reflexivity.

For transitivity $r = s$, $s = t$, a nonzero middle source is already allowed. If s is literal zero and one endpoint is literal zero, a premise gives the conclusion. Otherwise reverse $0 = t$ and join $r = 0$, $t = 0$ by the represented-zero lemma. Thus all numerical equality algebra is available. For

$F(v) \equiv A(v) = B(v)$, use numerical congruence on both sides and join with $A(s) = B(s)$ to obtain $A(t) = B(t)$. □

The proof uses weakening, cut, renaming, and only source-preserving helper substitutions. The one fresh variable used as a transport source in the induction step is never replaced by literal zero inside that derivation. Arbitrary targets are supplied by the induction or case rule itself. This distinction is necessary for a noncircular proof.

5 Completeness of the combined presentation

Lemma 5.1 (Equation induction inside $\mathcal{U}^-$)**.** *Contextual EI is derivable in $\mathcal{U}^-$ without using transport.*

Proof. Given a progressive step for $E(y)$, take an unused witness coordinate x. Atomic DI with witness 0 gives $D_{x,y}E(y)$. DE opens $Q_yE(y)$ independently of its fresh witness, and the zero-source branch of SE applies it to the supplied base $E(0)$ at the desired target. □

This lemma may use an *unguarded auxiliary* matrix E. It is why the proof language was stated explicitly. Applying Theorem 4.2, with additional retained auxiliary assumptions treated as passive context, restores all equation-template transport in $\mathcal{U}^-$ before any logical padding is needed.

Let $\mathcal{G}$ denote the inherited guarded-core calculus: the same public guarded formulas, the same deterministic introduction, and unrestricted transport on canonical templates, with anchored Q resources opened by DE. The repository supplies effective translations, both formula composites, and a whole-domain bi-interpretation of $\mathcal{G}$ with iPRA [3].

Lemma 5.2 (Transport on every canonical template)**.** *For a canonical template $F(v)$, the unrestricted rule $s = t, F(s) \vdash_{\mathcal{U}^-} F(t)$ is admissible.*

Proof. Induct externally on the formula template, uniformly in retained contexts. The equation case was just established. Suppose $F(v) = D_{x,y}B(v, x, y)$. Open $F(s)$ at a fresh witness a.

If B is guarded atomic, computation and the restored equality algebra prove $B(s, a, 0)$. Its opened Q resource therefore yields $B(s, a, Sz)$. Equation transport gives $B(t, a, Sz)$. Apply the deterministic atomic introduction to the target at witness a, ignoring its local assumption $B(t, a, z)$, and close DE.

If B is compound, introduce the target by the deterministic case rule. Use the smaller-template induction hypothesis and $t = s$ to turn its local base $B(t, a, 0)$ into $B(s, a, 0)$. Apply the source Q resource at Sz, and use the induction hypothesis forward to obtain $B(t, a, Sz)$. Close case introduction and DE. All newly chosen parameters are fresh for the retained equality and context. Passive substitutions preserve the guard. □

In the inherited clause presentation, the canonical formulas in Γ are read as ordinary assumptions, represented by their constant clauses.

Theorem 5.3 (Exact public consequence relation)**.** *For every finite context Γ of canonical formulas and every canonical A,*

$$\Gamma \vdash_{\mathcal{U}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{G}} A.$$

Proof. A guarded-core proof translates to $\mathcal{U}^-$: its computations, hypotheses, introductions, and resource eliminations are available, while each unrestricted canonical transport is replaced by Lemma 5.2. Starting from public formula contexts, the reachable clauses are constant clauses for ordinary assumptions and anchored clauses opened by DE. The former correspond to hypotheses, and each use of the latter corresponds to a Q elimination. No arbitrary additional clause context is being identified with a Q context.

Conversely, auxiliary soundness translates a $\mathcal{U}^-$ proof to iPRA. Agreement of the two readings on the canonical context and conclusion gives its ordinary guarded reading. The inherited faithful guarded/iPRA translation, including its canonical formula composite, returns a guarded proof of the original sequent. This is an effective translation of finite proofs using the inherited translations. □

The converse does not identify all auxiliary formulas with their ordinary anchored readings. Only the public context and conclusion use the agreement lemma. Likewise, Lemma 5.2 does not assert full transport or full formula congruence for every unguarded auxiliary binder.

6 The two interpretations and both composites

The consequence theorem transfers an existing bi-interpretation, rather than merely transferring consistency. Here are its components and the exact facts needed from the inherited foundations.

6.1 The arithmetic bridge

The ordinary scalar theory T_{eq} is intuitionistic first-order logic with equality over the iterator terms, the two computation equations, coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, separation $0 = 1 \to \bot$, and contextual induction for literal equations. Its inherited construction derives predecessor, equality reflection into a Boolean term, quantifier-free induction, pairing with projections, and compilation of every fixed primitive recursive definition, in that order [4, 5].

The direction to iPRA interprets each fixed iterator body by primitive recursion. In the reverse direction, a description of a primitive recursive function is compiled to an iterator term; a counter and the current result can be packed into the scalar state. The translations prove both term round trips equal to the original terms. They are interpretations over the whole numerical domain with actual equality, not a restriction to standard numerals or a quotient by an external equivalence relation.

The pinned Lean modules `BiInterpretation`, `NativeBiInterpretation`, and their dependencies state the two directions and the round trips for the ordinary scalar and native theories. The guarded grammar and padding bridge remain written mathematics in the pinned research package. This report does not claim that the present composition has been checked by Lean.

6.2 The logical bridge and guarded padding

The inherited native core has equations, Q, and D. Write $\mathrm{sel}(v, A, B)$ for its formula selector, equivalent to A at $v = 0$ and to B at successors, and $\mathrm{gate}(v, A)$ for its formula gate, true at zero and equivalent to A at successors. Their equational leaves use the scalar selector C; syntactic padding

aligns compound shapes. The ordinary logical operations are macros, for example

$$\begin{aligned} \bot &:= 0 = 1, \\ A \to B &:= Q_y \,\mathrm{sel}(y, A, B), \\ A \wedge B &:= Q_y \,\mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)), \\ \exists x\, A(x) &:= D_{x,y} \,\mathrm{gate}(y, A(x)), \\ A \vee B &:= \exists v \;\mathrm{sel}(v, A, B), \\ \forall x\, A(x) &:= A(0) \wedge Q_y A(y), \end{aligned}$$

where the parity iterator is a fixed scalar term. All natural deduction rules for these macros are inherited derived rules; they are not additions to Section 2.

To pass explicitly to the equivalent one-object-binder presentation, recursively replace each $Q_y B$ by $D_{u,y} B$ with a fresh unused u. The two are interderivable in the old core: paired elimination gives Q from the dummy-witness D, while case introduction and QE give the reverse direction with witness 0. After this expansion, Q is needed only as a context resource. For an equation E, put

$$G_v E = (C(v, 0, s) = C(v, 0, t)) \quad (E \equiv s = t), \qquad L(E) = D_{u,v} G_v E,$$

with fresh u, v, and let $P(E) = L(E)$, $P(A) = A$ at compound formulas. Guarded padding gives $P(A) \dashv\vdash A$. The translation from the unrestricted one-binder core to the guarded grammar is

$$T(s = t) = s = t, \qquad T(D_{x,y} B) = D_{x,y} P(TB).$$

It commutes with numerical substitution up to renaming. The inherited guarded-core argument proves both $JT(A) \dashv\vdash A$ in the old core and $TJ(B) \dashv\vdash B$ for canonical guarded B, where J is inclusion. The guarded atomic case of the second composite uses its true base; it is not an unrestricted congruence assertion for all auxiliary matrices.

Theorem 6.1 (Bi-interpretability and equiconsistency)**.** *The proposed public presentation, realized by* (O)*, is bi-interpretable with full first-order iPRA. Both numerical domains are whole and both numerical composite isomorphisms are given by $x = y$. Both formula composites are provably equivalent to the original formulas. In particular,*

$$\varnothing \vdash_{\mathcal{U}^-} 0 = 1 \quad \Longleftrightarrow \quad \mathrm{iPRA} \vdash \bot.$$

Proof. Let Φ be the inherited iPRA-to-guarded translation, obtained by composing the scalar compiler, the native logical translation, the dummy-witness expansion of Q, and guarded padding. Let Ψ be the guarded ordinary reading followed by the scalar-to-iPRA interpretation. Use these same maps for the new presentation. Theorem 5.3 preserves exactly every canonical consequence used by Φ, Ψ, and their guarded composite proofs.

The inherited iPRA composite proves $\varphi \leftrightarrow \Psi\Phi(\varphi)$ for every ordinary formula φ. The inherited guarded composite proves $A \dashv\vdash \Phi\Psi(A)$ for every canonical A; the consequence theorem makes this a proof in the proposed presentation. On numerical domains nothing has changed, and inherited term round-trip equations witness the literal identity graph. Finally the designated contradiction has the same reading throughout, giving the displayed equivalence. □

This proof uses the inherited full bridge as a proved input, with its sources supplied for inspection. A new derivation of predecessor or a new primitive-recursion compiler is neither needed nor claimed.

7 Standalone checkers, measurements, and tests

7.1 Exactly what is counted

Each selected checker is a complete Python module, including JSON input and the acceptance output. It imports no project file, producer, theorem table, normalizer, or stored proof database. On input a finite JSON array, it takes the final element as the goal and consumes instruction packets from right to left. An unused prefix is inert. Acceptance is successful process exit with exactly `ACCEPT` followed by a newline. Exceptions and nonzero exits reject. The programs do not rely on `assert`, and the supplied checks run both normally and under `python3 -O`.

Sizes use CPython 3.12.14. The counted source is

```
ast.unparse(ast.parse(source)) + '\n '.
```

Tokens exclude comments, encoding markers, whitespace/newline tokens, indent/dedent, and end marker. AST nodes are all nodes visited by `ast.walk`; bytes are UTF-8 bytes of the same normalized module. Gzip uses compression level 9 and `mtime=0`; the recorded zlib version is 1.3.2. Exact sources and machine-readable measurements are in `implementation/metrics.json`.

7.2 The compact representation

The arity codec is inherited and included as `reference/foundations/inherited_arity_checker.tex`. Its significant formula arities are one for D, two for equality, and three for Q. Canonical zero is JSON `0`, and a successor is a unary array. The iterator is a ternary array. Protected bound-coordinate slots include dummy slots so that one opening traversal increments its depth by $5 - \text{arity}$; at numerical roots the depth is three.

An instruction packet has four fields $[m, s, u, t]$. The equality/Q elimination test requires $m = [s, u]$ when s is truthy and $m = [t, t, t]$ when it is falsy. These are exactly the two branches of SE on canonical syntax. The target instance is checked before using the source instance. The remaining cases export an opened Q resource, introduce a computation equation, or introduce a paired formula. The root determines its local case/progress assumption.

The byte and AST profiles use a local fresh-name supply and a retained neutral truth. The token profile uses the inherited global fresh-name supply. Their certificates therefore differ; they are not three textual spellings guaranteed to accept the same raw JSON array. The supplied producer emits both protocols.

Raw aliases are interpreted as in the inherited codec. In particular, surplus formula arities denote truth. The original parity test `not len(a) % 2` can consequently be replaced by `len(a) == 2`: meaningful formulas retain their rule, and surplus truth annotations may use the case branch. This is the small implementation cleanup in addition to the mathematical composition. The inherited arithmetic-supply argument specializes fresh parameters only after translation into ordinary arithmetic, where that substitution is valid.

Some computation packets exclude a literal-zero initializer or a literal-zero successor predecessor. Use the certified represented zero $c_0 = I(2, 1; u.0) = 0$ to expand those computations: the normalized instance is an admitted computation, and transport at source c_0 returns the original instance. This needs no forbidden literal-zero transport and therefore does not reintroduce the bootstrap cycle.

7.3 The 522-byte module

The complete normalized byte profile is shown below. Its line wrapping in this PDF is only typesetting; its measured file is supplied unchanged.

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a
    < d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a
    := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) == 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

For direct use with untrusted end statements, prefer `code/checker_strict.py`. That single file also verifies the canonical closed guarded goal grammar. The three compact modules are appropriate under the same canonical-goal contract used for the pinned compact guarded-core results.

7.4 Executable evidence and its limits

The supplied reproduction script regenerates certificates and measurements. The recorded run has **150 process executions: 92 expected acceptances, 58 expected rejections, and zero mismatches**. It tests all three compact profiles with and without Python optimization, and tests the strict checker on public and nonpublic goals.

Positive cases include both computations, reflexivity, guarded atomic introduction, compound case introduction, implication elimination, guarded induction with a used induction hypothesis, and the new zero-source transformations under guarded D, a guarded auxiliary Q, and a compound D. Negative controls include the false goal $0 = 1$, missing or malformed instructions, a wrong computation endpoint, use of the removed atomic case mode, context leakage, and future-witness capture. The named proof audit records no remaining forbidden zero source and no incorrect introduction mode in the generated transformed proofs.

The producer is deliberately outside the trusted checker and is larger. It refuses to extend the guarded transport conversion to arbitrary unguarded atomic auxiliary templates. Finite test runs are implementation evidence, not a proof of soundness for every raw input or a proof-assistant verification of the universal metatheorems. The mathematical argument uses the inherited codec soundness and the written proofs above. Mathematical completeness, as for the baseline, abstracts from any particular machine's finite time, memory, and Python recursion limits.

8 What became simpler, and what remains open

The proved reduction is precise. Equality elimination and conditional elimination share the source-selected rule SE. Paired introduction has one root-determined local assumption. Their simultaneous use is justified by an equational theorem that does not depend on logical case introduction. All three measured complete compact profiles improve the corresponding pinned profiles, and the strict public-goal checker also becomes smaller.

The arithmetic term language, the public guarded grammar, the scalar primitive-recursion compiler, and the full logical interpretation remain inherited. The result is a modest reduction, not a significant new arithmetic basis or evidence of an absolute optimum. In particular, the argument does not show that the calculus remains complete when all noncanonical intermediate formulas are forbidden.

Further research questions

1. **Formalize the dependency theorem separately.** Introduce the purely equational restricted-source relation in Lean, prove unrestricted transport admissible there, and then instantiate that theorem in the guarded auxiliary calculus. This should make the absence of a logical bootstrap cycle explicit in the formal dependency graph.
2. **Can auxiliary unguarded induction be eliminated?** Either construct a direct restricted-source proof entirely within the canonical guarded language, or find an invariant showing that the present auxiliary extension is essential. The current proof does not decide this.
3. **Can one checker improve all three old minima at once?** The supplied profiles make separate tradeoffs. A joint improvement must count the entire module and preserve the same public-input contract.
4. **Can strict syntax validation share more of the kernel?** The fully counted strict checker is 1,202 bytes. A substantial improvement under that contract would be more informative than a further one-byte change in a permissive compact profile.
5. **Can zero-source proof expansion be shortened?** The current compilation uses a finite singleton test and an induction invariant. A simpler uniform derivation would shorten certificates and the formalization even if the checker itself did not become smaller.

A Reproduction and archive contents

From the extracted archive root, run:

```
python3 implementation/reproduce_composition.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The first command uses only Python's standard library. The two LaTeX runs require an ordinary TeX installation with the packages named in the source. The archive contains:

- `report.tex` and the compiled `report.pdf`;
- four standalone modules under `code/`, with corresponding normalized gzip files and matching copies in the reproduction directory;
- the producer, named proof audits, generated JSON certificates, positive and negative controls, metrics, and detailed process receipts under `implementation/`;
- frozen mathematical foundations and exact comparison sources, with source provenance and the upstream MIT-0 license;
- `CLAIMS.json`, `README.txt`, and a file checksum manifest.

The checker modules do not import any of the other archive components.

B A substitution distinction in the baseline

The pinned README describes direct numerical substitution as unavailable in the source-restricted kernel. The precise distinction is useful here: substitution cannot simply map every restricted transport node to another one, because a variable source may become literal zero. Once an exact sequent-equivalence theorem with the unrestricted kernel has been proved, substitution is nevertheless *admissible as a derived operation* by translation to that kernel, substitution there, and translation back. The equational proof in this report does not use that conclusion to prove its own source-elimination theorem.

References

[1] Vladimir Reshetnikov and ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at the pinned commit.
https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md

[2] ProveIt, *source_dispatch*, especially `proofs/zero_source_admissibility.tex`, the complete checkers, and the certificate normalizer, at the same commit. The represented-zero joining argument, singleton test, filter, and zero-source reversal are inherited from this package.
https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/source_dispatch

[3] ProveIt, *guarded_core*, especially `sections/guarded_core.tex`, the public canonical grammar, padding translations, both composites, and raw auxiliary reading.
https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/guarded_core

[4] ProveIt, *certified_zero/reference*: inherited choice basis, ordinary bridge, scalar predecessor, equality reflection, primitive recursion, and arity checker. Frozen copies are included under `reference/foundations/`.
https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/certified_zero/reference

[5] ProveIt, *ConciseIPRA Lean sources*, including `BiInterpretation`, `NativeBiInterpretation`, and `NativeZeroSource`, at the pinned commit. These are upstream results; the present composition is not an additional Lean-verified theorem.
https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA